FIELD MANUAL

VERSION 1.1 · CARETAKER-COMPLIANT · AMENDMENTS PACK

Universal Governance Spec

Local Model Deployment

> You are not adding governance to a system.
> You are placing the system INSIDE governance.
> All mutations pass through the kernel. No side paths. No trusted shortcuts.

COLOR SYSTEM

BLUE	Architecture / structural sections
RED	Failure modes / MUST NOTs / adversarial
GOLD	Invariants / mandatory / final principles
PURPLE	Advanced / extended / human-system

FRONT MATTER

Purpose & Operating Principle

Purpose

This manual defines ALL required knowledge, systems, and operational practices necessary to deploy the Universal Governance Execution Kernel on local AI systems. It is exhaustive, implementation-oriented, and strictly aligned with the locked specification.

Operating Principle

- You are not adding governance to a system.
- You are placing the system INSIDE governance.
- All mutations must pass through the kernel.
- There are no side paths, bypasses, or “trusted shortcuts.”

INVARIANT · Governance is the execution layer

The kernel is the only path through which state changes are allowed.

If anything can mutate state outside it, the system is invalid.

How to read this manual

Each numbered section opens its own page with a colored band. Blue marks architecture and structural sections. Red marks failure modes, prohibitions, and adversarial conditions. Gold marks invariants, mandatory rules, and final principles. Purple marks the extended and human-system layers (sections 12–50).

Verbatim specification text is preserved exactly as authored. Implementation-credibility expansions are appended inline as EXPANSION blocks demarcated with gold rules — they add worked examples, reference tables, and verification checklists without altering the locked spec.

v1.1 NOTICE · Amendments Pack v1.1 is appended after §50

Twelve normative amendments (§§A-1 through A-12) close gaps identified in the v1 implementation review.

Amendments take precedence over earlier text wherever they touch a previously-described primitive.

v1 content is preserved verbatim; nothing has been deleted.

§1 Foundational Knowledge Domains

You must understand the following disciplines at an implementation level:

1.1 Distributed Systems

- Deterministic processing
- Idempotent operations
- Event-driven architecture
- Failure isolation
- Replayability

1.2 Applied Cryptography

- SHA256 hashing (canonical JSON)
- Ed25519 signatures (sign/verify)
- Key management (generation, rotation, revocation)
- Integrity verification chains
- Non-repudiation

1.3 API Systems

- RESTful design (strict contracts)
- Schema validation (JSON schema enforcement)
- Request/response determinism
- Idempotency keys (traceId usage)

1.4 Data Persistence

- Append-only storage models
- SQLite / Postgres fundamentals
- Transaction integrity
- Write-ahead logging (WAL)
- Consistency guarantees

1.5 Policy Systems

- Rule engines (deterministic evaluation)
- Conflict resolution (deny-by-default)
- Policy scoping (resource, actor, operation)
- Risk-tier modeling

1.6 Identity & Authorization

- Capability-based security (not role-only)
- Token scoping and constraints
- Delegation attenuation
- Token introspection

1.7 Observability & Auditing

- Event sourcing
- Trace correlation (traceId propagation)
- Deterministic replay systems
- Audit integrity guarantees

1.8 Concurrency Control

- Race condition prevention
- Locking strategies (optimistic/pessimistic)
- Ordering guarantees for ledger writes
- Timestamp vs logical ordering

EXPANSION · Reference standards every implementer must cite

These are not suggestions; they are the authoritative source documents the kernel must conform to. Implementation reviews should verify each control against the cited standard.

Standard	Subject	Where it applies
FIPS 180-4	SHA-256 hashing	Hash chain integrity, payloadHash, recordHash
RFC 8032	Ed25519 EdDSA signatures	DecisionRecord, PER, approval signing
RFC 8785	JSON Canonicalization Scheme (JCS)	Canonical JSON for all hashed payloads
RFC 7519	JWT	CapabilityToken envelope (claim set, expiry)
NIST SP 800-207	Zero Trust Architecture	Capability-based access, no implicit trust
NIST SP 800-53	Security & Privacy Controls	AU-* audit, AC-* access, SC-* comm.
RFC 5905	NTPv4	Time synchronization for token / quorum expiry
RFC 6749 / 8252	OAuth 2.0 / Native Apps	Delegation attenuation patterns

§2 System Architecture Overview

All local deployments MUST implement the following components:

2.1 Governance API Layer

Endpoints (fixed, no additions):

```
POST /governance/evaluate
POST /governance/decisions
POST /capabilities/issue
POST /capabilities/introspect
POST /capabilities/revoke
POST /observability/events
GET  /observability/traces/{traceId}
POST /observability/traces/{traceId}/replay
```

2.2 Enforcement Kernel

Implements:

- Invariants
- Evaluation pipeline
- Quorum engine
- Decision synthesis

2.3 DecisionRecord Ledger

- Append-only
- Hash-linked
- Persisted (SQLite/Postgres)
- Rejects chain breaks

2.4 CapabilityToken Authority

- Token issuance
- Token validation
- Revocation handling
- Delegation enforcement

2.5 Quorum Engine

- Approval aggregation
- Signature validation
- Threshold enforcement
- Governance evaluation per approval

2.6 Observability System

- Event emission
- Immutable payload storage

- Trace reconstruction
- Replay engine

2.7 Compensation Engine

- Forward-only corrective mutations
- Binding to original decision
- Policy + quorum governed

§2 · CONT.

Endpoint Contract (Canonical Reference)

Each endpoint enforces a strict request/response schema. Idempotency is keyed on traceId; duplicate submissions return the original DecisionRecord, never a divergent one.

Verb	Path	Request	Response	Idem. key
POST	/governance/evaluate	PER	PolicyEvaluationResult	traceId
POST	/governance/decisions	DecisionRecordSubmit	DecisionRecord	traceId
POST	/capabilities/issue	TokenIssueRequest	CapabilityToken	requestId
POST	/capabilities/introspect	{token}	TokenIntrospection	—
POST	/capabilities/revoke	RevokeRequest	RevokeReceipt	requestId
POST	/observability/events	ObservabilityEvent	Ack(eventId)	eventId
GET	/observability/traces/{traceId}	—	TraceBundle	—
POST	/observability/traces/{traceId}/replay	ReplayReq	ReplayResult	replayId

§3
Governance Execution Flow (Mandatory)

Every mutation MUST follow EXACTLY:

Step 1 — Construct PER

- Include actor, action, resource, mutation, traceId
- Compute inputHash (canonical JSON)

Step 2 — Call /governance/evaluate

- Submit PER
- Receive PolicyEvaluationResult

Step 3 — Evaluate Outcome

- allow → proceed
- deny → stop
- escalate → wait for quorum

Step 4 — If quorum required

- Collect QuorumApproval(s)
- Each approval must:
- Have capabilityToken
- Be governance-evaluated
- Be signed (ed25519)

Step 5 — Write DecisionRecord

- Include:
- inputHash
- outputHash
- quorum data
- Compute recordHash (canonical JSON)

Step 6 — Emit Observability Event

- payloadHash must match canonical payload
- payloadRef must be immutable

Step 7 — Create MutationGovernanceBinding

- Bind token + evaluation + decision + quorum

Step 8 — Execute Mutation

- Only after binding verification

MANDATORY · No step may be skipped or reordered.

A binding that omits any predecessor is invalid by definition. The kernel rejects it before the mutation reaches the resource adapter.

§3 · CONT.

Worked Example — Governed Mutation, End-to-End

A local model proposes resource.user.email change. The proposal is wrapped, evaluated, quorum-approved, recorded, and only then committed:

```
1) PER (canonical, sorted keys):
   {"action":"update","actor":{"id":"agent:llm-7","type":"model"},
    "mutation":{"after":{"email":"a@b.io"},"before":{"email":"a@b.com"}},
    "resource":{"classification":"restricted","id":"user:42"},
    "schemaVersion":"1","traceId":"01HX...QM"}
   inputHash = sha256(canonical_json(PER))

2) POST /governance/evaluate -> PEResult{outcome:escalate, requiredQuorum:2}

3) Collect 2 QuorumApproval rows, each ed25519-signed by approver key,
   each with valid CapabilityToken (scope=approve:user.update,
                                    riskTier<=restricted).

4) DecisionRecord:
   {"inputHash":"...","outputHash":"...","outcome":"allow",
    "prevRecordHash":"<H_n-1>",
    "quorum":{"aggregatedHash":"...","approvals":[...]},
    "schemaVersion":"1","timestamp":"2026-04-27T10:08:00Z",
    "traceId":"..."}
   recordHash = sha256(canonical_json(DecisionRecord))

5) ObservabilityEvent emitted with payloadHash matching canonical payload.
6) MutationGovernanceBinding created:
     token + PEResult + DecisionRecord + quorum.
7) Adapter writes the new email ONLY after binding verification succeeds.
```

§4
Data Integrity Requirements

4.1 Canonical JSON

- All hashes use canonical JSON
- No field ordering variation
- No omitted fields

4.2 Hash Chain

- Each DecisionRecord links to previous
- Chain break = system halt

4.3 Determinism

- Same input MUST produce same output
- Required for replay validation

EXPANSION · Canonical JSON rules (RFC 8785 / JCS-aligned)

Implementations should treat canonicalization as a single-purpose, well-tested module with no per-call options. Drift here breaks every hash downstream.

- Sort all object keys lexicographically (codepoint order).
- Encode UTF-8; no BOM; escape only required JSON control characters.
- Numbers: shortest round-trippable form; no trailing zeros; no exponent unless required.
- Booleans / null: lowercase literals only.
- Arrays: preserve order (semantic); never sort.
- No whitespace anywhere; no comments; no duplicate keys.

```
# Reference implementation contract (Python)
import json, hashlib
def canonical(obj):
    return json.dumps(obj, sort_keys=True, separators=(',',':'),
                      ensure_ascii=False, allow_nan=False).encode('utf-8')
def sha256_canon(obj):
    return hashlib.sha256(canonical(obj)).hexdigest()
```

§5
Capability Token System

You must implement:

- Issuance (signed by governanceAuthority)
- Introspection (runtime validation)
- Revocation (immediate effect)

Validation MUST check:

- Signature validity
- Expiry
- Scope (operation/resource)
- Constraints (riskTier, jurisdiction)

MANDATORY · No token = no mutation.

Every governed mutation must reference a valid, introspected, non-revoked CapabilityToken bound to the actor and scoped to the operation.

EXPANSION · CapabilityToken envelope (reference)

```
{
  "schemaVersion": "1",
  "tokenId":   "tok_01HX...",
  "issuer":    "governanceAuthority/v3",
  "subject":   {"id":"agent:llm-7","type":"model"},
  "scope":     ["update:user.email","read:user.*"],
  "constraints": {
     "riskTier":     "restricted",
     "jurisdiction": "us-only",
     "rateLimit":    "60/min"
  },
  "delegation": {"depthMax": 2, "attenuationOnly": true},
  "issuedAt":  "2026-04-27T10:08:00Z",
  "notBefore": "2026-04-27T10:08:00Z",
  "expiresAt": "2026-04-27T10:38:00Z",
  "kid":       "auth-key-2026-04",
  "sig":       "<ed25519 over canonical body w/o sig>"
}
```

§6
Policy Engine

Policy engine MUST:

- Evaluate PER against policyRefs
- Enforce deny-by-default
- Resolve conflicts deterministically
- Output structured decision basis

Policies control:

- Allowed operations
- Required quorum
- Risk tier
- Resource access

EXPANSION · Risk tier × classification × quorum mapping (reference)

This table is the canonical mapping the policy engine must enforce. Adjust thresholds in the policy version, not in code.

Classification	Mutation kind	Risk tier	Min approvals	Quorum rule
public	softMutation	low	0	single signer, single approval
internal	softMutation	low	0	single signer
internal	hardMutation	medium	1	1 approver, role:Operator
restricted	hardMutation	high	2	2 approvers, distinct roles
restricted	governanceMutation	high	2	2 approvers, must include PolicyAuthority
critical	any	critical	3	3 approvers, distinct roles, <15 min window

§7 Quorum System

Required for governanceMutation (and some hardMutation):

You must support:

- Role-based approval validation
- CapabilityToken per approver
- Governance evaluation per approval
- Signature verification
- Deduplication (first valid approval wins)

Failure conditions:

- Expired quorum window
- Invalid signatures
- Insufficient approvals

§8
DecisionRecord Ledger

Requirements:

- Append-only
- Hash-linked
- Durable storage
- Cryptographically signed

Must include:

- inputHash
- outputHash
- prevRecordHash
- quorum.aggregatedHash
- recordHash

INVARIANT · Ledger is the source of truth.

Every other store is derived. If a derived store disagrees with the ledger, the derived store is wrong; never the other way around.

EXPANSION · DecisionRecord schema (reference)

```
{
  "schemaVersion": "1",
  "recordId": "drec_01HX...",
  "traceId":  "01HX...",
  "inputHash":  "<sha256 of PER>",
  "outputHash": "<sha256 of canonical decision body>",
  "outcome":    "allow|deny|escalate|compensate",
  "policyRefs": ["policy:user.update@v3"],
  "quorum": {
    "approvals": [ {"approverId":"...","tokenId":"...","sig":"<ed25519>"} ],
    "aggregatedHash": "<sha256 of canonical approvals[]>"
  },
  "prevRecordHash": "<sha256 of prior record canonical body>",
  "timestamp": "RFC3339 UTC, monotonic",
  "authoritySig": "<ed25519 over canonical body w/o this field>",
  "recordHash":   "<sha256 of canonical body INCLUDING authoritySig>"
}
```

§9 Observability & Replay

You must implement:

9.1 Event Emission

- Every step emits event
- Must include traceId

9.2 Immutable Storage

- payloadRef points to immutable storage
- payloadHash validates content

9.3 Replay Engine

Modes:

- verify → must match ledger
- simulate → no writes

Replay MUST:

- Recompute hashes
- Detect mismatches
- Emit error events on divergence

§10
Compensation System

INVARIANT · Compensation is NOT rollback.

It is a NEW mutation, forward-only, linked to the original decision.

It is:

- A NEW mutation
- Forward-only
- Linked to original decision

Requirements:

- Must pass governance evaluation
- Must reference originalDecisionRecordId
- Must include originalRecordHash
- Must produce DecisionRecord(outcome=compensate)

§11
Security Model

You must enforce:

- Zero trust (no implicit trust)
- Capability-based access
- Signed approvals
- Signed decisions
- Immutable logs

Attack surfaces to protect:

- Token forgery
- Approval spoofing
- Ledger tampering
- Replay manipulation

LOCAL MODELS

Local Model Integration

For local LLMs (GPT, LLaMA, Mistral, etc.):

ZERO TRUST · The model is NOT trusted.

It is treated as subject.actorType = model. The model generates intent and DOES NOT execute mutations.

All outputs must be:

- Wrapped into PER
- Evaluated before execution

FAILURE

Failure Handling

You must handle:

- Evaluation failure → deny
- Token invalid → deny
- Quorum timeout → deny
- Ledger inconsistency → halt system
- Replay mismatch → raise error event

FAIL-CLOSED · System must fail CLOSED.

No partial execution. No retry-until-success. The denial itself is recorded as a DecisionRecord and emitted to observability.

EXPANSION · Failure → Detection → Response matrix

Failure	Detection	Response
Token forgery	Signature verify fail	deny + revoke + alert security
Token replay	tokenId already bound	deny + record violation
Approval spoof	Approver signature fail	deny + invalidate quorum window
Ledger chain break	prevRecordHash mismatch	halt all mutations + page on-call
Replay divergence	Recomputed hash differs	emit error event + auditor flag
Storage failure	Write or fsync error	deny; queueing is forbidden
Clock drift	ts > leeway (NTP fail)	deny + force re-sync + alert
Quorum timeout	window expired	deny + DecisionRecord(outcome=deny)

PERFORMANCE

Performance Considerations

You should design for:

- Low-latency evaluation
- Efficient hashing
- Indexed ledger queries
- Parallel quorum collection

But NEVER at the cost of:

- determinism
- auditability
- correctness

DEPLOY

Deployment Checklist

Before going live:

☐	Governance endpoints operational
☐	Ledger persistence verified
☐	Hash chain integrity tested
☐	Replay engine validated
☐	Token authority initialized
☐	Quorum policies configured
☐	Observability pipeline active

EXPANSION · Pre-flight verification (signable)

Each row should be signed-off by a named operator and dated. Keep this page in the runbook.

☐	Governance Authority root keypair generated; public key distributed; private key in HSM/TPM
☐	Initial CapabilityToken issued to Operator role; introspection live
☐	Policy set v1 loaded; deny-by-default verified against negative tests
☐	Genesis DecisionRecord written: prevRecordHash = 0, signed, recordHash captured
☐	Append-only ledger storage (WAL on, fsync per record) confirmed
☐	Observability pipeline emits + stores immutable payloadRef; payloadHash check passes
☐	Replay engine verify-mode against last 1,000 records: zero divergence
☐	NTP time source synced; max drift < 250ms across nodes
☐	Quorum thresholds reflect risk-tier × classification matrix
☐	Resource adapters reject any direct write that lacks MutationGovernanceBinding
☐	Backup target tested with hash-chain integrity verification on restore
☐	Chaos test: token revocation mid-flight → mutation correctly denied

PROHIBITIONS

What You MUST NOT Do

- Do NOT bypass /governance/evaluate
- Do NOT mutate without DecisionRecord
- Do NOT allow unsigned approvals
- Do NOT modify ledger entries
- Do NOT introduce new primitives

PROHIBITION · These are violations, not warnings.

Any of the above is a system invariant breach and must trigger immediate halt + escalation. The system is invalid until reverified.

MENTAL MODEL

Mental Model

Think of the system as:

- A blockchain-like decision ledger
- A policy firewall for all mutations
- A cryptographic audit system
- A deterministic execution engine

FINAL STATE

Final State (Core)

If implemented correctly:

- Every action is governed
- Every decision is provable
- Every mutation is reversible (via compensation)
- Every failure is detectable
- Every system is auditable

END OF FIELD MANUAL

EXTENDED

Additional Required Knowledge — Field Manual Supplement

This section defines advanced, non-optional operational knowledge required to successfully deploy, secure, and sustain the Universal Governance Spec (v1) in real-world local-model environments.

These are NOT new primitives. These are REQUIRED operational disciplines derived from the governing system.

Each section below opens its own page. The cumulative requirement is that all sections 12–32 (operational layer) and 33–50 (human + system layer) are implemented alongside the core spec for a deployment to qualify as caretaker-compliant.

§12
Cryptographic Key Management (Mandatory)

Governance Authority Keys

- ed25519 keypair REQUIRED for:
- DecisionRecord signing
- PolicyEvaluationResult signing
- CapabilityToken issuance
- Private keys MUST:
- Never leave secure boundary (HSM, TPM, enclave, or equivalent)
- Be non-exportable when possible
- Public keys MUST:
- Be distributed to all verification nodes
- Be versioned and rotated

Key Rotation

- MUST support rotation without breaking verification
- Old keys MUST remain valid for historical verification
- Rotation events MUST emit ObservabilityEvent(type=capability.issued or equivalent)

Compromise Handling

- Immediate revocation
- Emit observability event
- Invalidate affected tokens
- Flag replay verification anomalies

EXPANSION · Key lifecycle reference

Stage	Mechanism	Operational note
Generate	HSM / TPM / sealed enclave	FIPS 140-2 L3+ where available
Distribute (pub)	Signed key bundle, multi-channel	Pin in adapters; version=N
Use	Sign DecisionRecord, PER, tokens	kid claim required
Rotate	Issue v=N+1, accept N and N+1	Emit capability.issued event
Retire	Stop signing with N; verify only	Keep N for historical verification
Revoke	Compromise: invalidate N + tokens	Cascade: re-verify recent records

§13
Canonical JSON Standardization (Critical)

All hashing depends on canonical JSON.

Canonicalization MUST:

- Sort keys lexicographically
- Remove whitespace variability
- Enforce consistent encoding (UTF-8)
- Normalize numbers, booleans, nulls

Failure results in:

- Hash mismatch
- Replay failure
- Ledger invalidation

CRITICAL · This is a system-critical dependency.

See §4.1 expansion for the reference implementation contract. Treat the canonicalizer as fixed; never 'optimize' per call site.

§14
Time Synchronization (Strict Requirement)

All timestamps affect:

- quorum expiry
- token expiry
- replay determinism

System MUST:

- Use synchronized time source (e.g., NTP with fallback)
- Enforce monotonic progression where required

Clock drift risks:

- false quorum failure
- token misvalidation
- replay divergence

EXPANSION · Time discipline thresholds

Recommended budgets — deployments may tighten, never loosen, without justification.

Discipline	Budget	Action on breach
Inter-node clock skew	< 250 ms	Halt writes if exceeded
Token expiry leeway	< 5 s	Deny on violation
Quorum window default	5–15 min	Per risk tier; critical < 15 min
Replay timestamp tolerance	Exact match	Use logical ordering for ties

§15
State Hashing Strategy (Mandatory)

Every governed resource MUST support deterministic state hashing.

Requirements:

- Same state → same hash
- No nondeterministic fields (timestamps, random IDs unless excluded)
- Hash boundary MUST be clearly defined

Used for:

- beforeHash validation
- afterHash verification
- compensation correctness

§16

Resource Adapter Layer (Required for Local Models)

Local systems MUST implement adapters that:

Translate:

- model actions → proposedMutation
- system state → beforeHash/afterHash

Enforce:

- no direct mutation bypass
- all writes go through governance pipeline

ARCHITECTURE · Adapters are enforcement chokepoints.

If a write path exists outside the adapter, it is the bypass. Inventory every storage write and confirm each one ends at an adapter call.

§17
Failure Modes (Fail-Closed Requirement)

System MUST default to DENY on:

- token introspection failure
- policy evaluation failure
- quorum uncertainty
- hash mismatch
- storage failure
- observability failure

FAIL-CLOSED · No partial execution allowed.

A 'mostly-applied' mutation is worse than a denied one. The kernel never half-commits.

§18
Idempotency + Retry Safety

All governance operations MUST be idempotent:

- Re-submitting same PER MUST NOT produce divergent outcome
- DecisionRecord must not duplicate inconsistently
- Quorum approvals must respect deduplication rules

Use:

- requestId
- evaluationId
- approvalId

to enforce uniqueness.

§19
Concurrency Control

Parallel mutations MUST:

- not violate hash chain integrity
- not overwrite shared state without detection

Mechanisms:

- optimistic locking (hash comparison)
- or serialized mutation execution

Conflict outcome:

- → DENY + DecisionRecord

§20
Storage Hardening

Ledger + Observability storage MUST ensure:

- append-only guarantees
- tamper resistance
- crash consistency

Recommended:

- WAL (write-ahead logging)
- checksums
- replication
- periodic integrity verification

Optional:

- Merkle tree anchoring
- external notarization

§21 Trace Propagation Discipline

traceId MUST:

- originate at entry point
- propagate through:
- PER
- PEResult
- DecisionRecord
- QuorumApproval
- ObservabilityEvent

Breakage = audit failure.

§22 Policy Engine Requirements

Policy system MUST support:

- deterministic evaluation
- conflict resolution (denyOnConflict)
- versioning
- traceable policyRefs

Policies MUST NOT:

- override invariants
- bypass quorum rules
- weaken capability enforcement

§23
Capability Token Enforcement (Deep)

Validation MUST include:

- signature verification
- expiry check
- status check (revoked/suspended)
- scope validation
- constraint validation

Delegation:

- MUST be attenuation-only
- MUST enforce depth limits

§24
Quorum Security Hardening

Approvals MUST:

- be individually signed
- be independently validated
- require valid capability token

System MUST prevent:

- replay of approvals
- duplicate approvals
- forged roles

§25
Compensation Safety Controls

Compensation MUST:

- verify current state via beforeHash
- never assume state correctness
- never cascade uncontrolled

Optional control:

- limit number of compensations per original record

§26
Replay System Validation

Replay MUST:

- recompute ALL hashes
- re-evaluate ALL policies (in verify mode)
- detect ANY divergence

Replay mismatch MUST:

- emit error event
- flag integrity violation

§27
Deployment Architecture (Local Model Environment)

Minimum components:

- Governance API layer (/governance/evaluate)
- Policy engine
- Token authority
- Ledger storage
- Observability storage
- Resource adapters

Optional separation:

- control plane vs execution plane

§28
Security Boundaries

Define strict boundaries between:

- model execution
- governance enforcement
- storage layers
- key management

No model may:

- self-issue tokens
- self-approve quorum
- bypass evaluation

§29 Testing Requirements

Must include:

- invariant violation tests
- replay determinism tests
- quorum edge cases
- token revocation scenarios
- compensation correctness

System is not valid without passing these.

§30 Operational Monitoring

Monitor:

- decision throughput
- denial rates
- quorum delays
- replay failures
- hash mismatches

Alert on:

- replay mismatch
- invalid signature
- ledger break attempt

§31
Backup + Recovery

Backups MUST:

- preserve hash chain integrity
- include ledger + observability

Recovery MUST:

- verify hashes before restore acceptance

§32
Zero-Trust Assumption

System MUST assume:

- every component can fail or be compromised
- verification is mandatory at every step

Trust is cryptographic, not positional.

MID-MANUAL

Final Condition (Operational Layer)

This knowledge is REQUIRED to make the Universal Governance Spec:

- actually enforceable
- actually secure
- actually deterministic
- actually auditable

Without these operational disciplines:

The specification remains correct, but the system becomes vulnerable, non-deterministic, or unverifiable.

This completes the field manual knowledge layer.

FINAL SUPPLEMENT

Real-World Execution + Human System Integration

Short answer: yes — there is still one final layer most people miss, and without it, real-world deployments fail even if everything else is correct.

Below is the last complete supplement. After this, you are truly at "nothing else missing" for a field manual.

This layer covers the gap between a perfectly designed governance system and a system that actually survives real-world use, operators, and adversarial conditions.

§33

Human Governance Model (Non-Optional)

You must define:

- Who can issue CapabilityTokens
- Who can approve quorum
- Who defines policy
- Who can trigger compensation
- Who audits the system

Required roles (minimum):

- GovernanceAuthority (root signer)
- PolicyAuthority (defines policies)
- QuorumApprover(s) (multi-party)
- Auditor (read-only, replay authority)
- Operator (runs system, no override power)

RULE · Separation of duties

No single role should be able to: issue token + approve quorum + define policy. That is a critical separation-of-duties control.

EXPANSION · Role × Capability matrix (least-privilege reference)

Role	Issue token	Define policy	Approve quorum	Audit/replay	Sign decisions
GovernanceAuthority	✓	—	—	—	✓ (sign root)
PolicyAuthority	—	✓	—	—	—
QuorumApprover	—	—	✓	—	—
Auditor	—	—	—	✓ replay/verify	—
Operator	—	—	—	—	✓ run only

§34
Bootstrap / Genesis State (Critical)

You must define how the system starts.

Required:

- First GovernanceAuthority keypair (root of trust)
- First CapabilityToken issuance (who gets initial authority)
- First DecisionRecord (genesis record with prevRecordHash = 0)
- Initial policy set

Without a clean genesis:

- the ledger cannot be trusted
- replay cannot be verified

§35

Policy Lifecycle Management

Policies are the ONLY allowed extension surface.

You must support:

- versioning (policy v1, v2, etc.)
- activation timestamps
- deprecation (but not deletion)
- traceability (policyRefs MUST map to versioned policy)

RULE · No retroactive policy changes

Policy changes MUST NOT retroactively alter past decisions. Replay uses current policy in verify mode (drift detection) and historical ledger as truth.

§36

Schema Versioning Strategy

All primitives already have schemaVersion.

You must define:

- how upgrades happen
- how backward compatibility is preserved
- how replay handles mixed versions

RULE · Old DecisionRecords MUST remain valid forever.

No breaking changes allowed.

§37
Adversarial Model Assumption

You must assume:

- malicious actors with valid tokens
- compromised services
- replay attacks
- forged inputs
- timing attacks

Controls already defined must be enforced strictly:

- signature validation
- capability scope enforcement
- deduplication
- hash verification

Never trust:

- inputs
- models
- operators

Only trust:

- cryptographic verification

§38
Data Classification + Policy Mapping

Your PER includes resource.classification.

You must define:

- what “public / internal / restricted / critical” means
- how policies differ per classification
- how riskTier is derived from classification

Example (must be defined in your system):

- critical → always quorum
- restricted → policy-dependent quorum
- public → no quorum

Without this:

- policy engine cannot function correctly

EXPANSION · Classification → derived risk tier (reference)

Classification	Definition (example)	Risk tier	Allowed mutation kinds
public	Public-facing data, no PII	low	softMutation only
internal	Org-only, non-sensitive	medium	soft + hard with quorum=1
restricted	PII, secrets, governance state	high	hard + governance, quorum≥2
critical	Root keys, ledger, genesis, kernel config	critical	always quorum, distinct roles, <15 min window

§39

Latency + Performance Tradeoffs

Governance adds latency.

You must design for:

- synchronous vs async quorum
- batching where allowed (never skip evaluation)
- caching token introspection (within validity window)

RULE · Performance optimizations MUST NOT bypass:

evaluation

decision recording

observability

§40
Offline / Degraded Mode Strategy

Define behavior when:

- governance service unavailable
- token introspection fails
- storage unavailable

Required behavior:

FAIL-CLOSED · No mutation allowed.

Optional: queue requests for later evaluation (but do not execute).

§41
Multi-Node Consistency

If distributed:

- all nodes MUST agree on ledger order
- prevRecordHash must be globally consistent

Use:

- leader-based write
- or consensus mechanism (Raft/Paxos-style)

Ledger divergence = system failure.

§42

Integration with Local Models

Critical rule:

> RULE · Models are UNTRUSTED.
>
> They propose mutations and NEVER execute mutations directly. All model outputs MUST be translated into PER. No exception.

§43
Prompt / Model Hardening

Even though governance enforces execution:

You must still:

- constrain prompts
- prevent instruction injection
- sanitize outputs

Reason:

Governance stops bad execution, but not bad proposals flooding the system.

§44
Rate Limiting + Abuse Control

Prevent:

- PER flooding
- quorum spam
- replay abuse

Controls:

- per-actor rate limits
- per-token limits
- anomaly detection

§45

Legal + Compliance Mapping (Optional but Practical)

Map system to:

- SOC2 (audit trail → DecisionRecord + Observability)
- GDPR (traceability + controlled mutation)
- NIST (zero trust + verification)

Not required for function, but required for real-world adoption.

EXPANSION · Compliance crosswalk

Standard	Requirement	How this kernel satisfies it
SOC2 CC7.2	System monitoring & anomalies	Observability events + alerts on §30
SOC2 CC8.1	Change management	DecisionRecord per mutation; policy versioning §35
GDPR Art. 30	Records of processing	Append-only ledger + traceId propagation §21
GDPR Art. 17	Right to erasure	Compensation §10 (forward-only erasure record)
NIST 800-207	Zero Trust Architecture	Capability tokens §5/§23, no implicit trust §32
NIST 800-53 AU-2/AU-9	Audit events / protection	Immutable observability storage §9.2 / §20
ISO 27001 A.12.4	Logging & monitoring	Hash-linked ledger + replay verification §26

§46
Documentation + Operator Training

Operators MUST understand:

- invariants cannot be bypassed
- compensation is forward-only
- denial is correct behavior

Most system failures come from:

- humans trying to “override” governance

That must be impossible.

§47
Chaos Testing (Highly Recommended)

Simulate:

- token revocation mid-flight
- quorum timeout
- hash mismatch
- storage failure
- replay mismatch

Verify:

- system always fails safely

EXPANSION · Chaos test catalog

ID	Scenario	Expected behavior
CT-01	Revoke token between evaluate and binding	deny + DecisionRecord(deny)
CT-02	Quorum approver offline; window expires	deny + DecisionRecord(deny, reason=quorum_timeout)
CT-03	Mutate prevRecordHash on disk	halt + alert; chain break detected on next read
CT-04	Inject duplicate approval (same approverId)	dedup; only first valid wins
CT-05	Storage fsync error during DecisionRecord write	deny; no partial commit; observability alert
CT-06	Replay verify on tampered payloadRef	error event emitted; integrity violation flag
CT-07	Clock skew node B by +90s	deny token-expiry edge cases; NTP alert
CT-08	Forge approval signature with wrong key	verify fail; deny; alert security

§48
System Boundary Definition

You must explicitly define:

What IS governed:

- all state mutations
- all model actions affecting state

What is NOT governed:

- read-only operations
- external systems (unless integrated)

Ambiguity here creates bypass risk.

§49
Upgrade / Migration Strategy

When deploying new versions:

- freeze writes if necessary
- verify ledger integrity pre/post
- maintain replay compatibility

Never:

- rewrite ledger
- alter historical hashes

§50

Final Principle — Governance Is the Execution Layer

This is the most important conceptual requirement:

The system you built is NOT:

- a logging system
- a policy checker
- an approval workflow

It IS:

> INVARIANT · the ONLY path through which state changes are allowed
>
> If anything can mutate state outside it, the system is invalid.

FINAL STATE

If you implement all three layers

If you implement:

- Core Spec (v1 LOCKED)
- Operational Layer (12–32)
- Human + System Layer (33–50)

Then you have:

- A fully enforceable
- fully auditable
- fully deterministic
- fully adversarially resilient
- governance execution system

CLOSING · There is nothing missing after this.

Anything further is optimization, scaling, or domain-specific policy — not foundational knowledge.

AMENDMENTS · V1.1

Locked-Spec Corrections — Caretaker-Compliant

This pack closes the twelve specification gaps identified during the v1 implementation review. Each amendment is normative: it tightens, fixes, or makes deterministic a behavior that the v1 core spec defined informally. The original v1 text is preserved verbatim earlier in this manual; the amendments below take precedence wherever they touch a previously-described primitive.

INVARIANT · Amendments are part of the locked spec

Implementations claiming v1.1 caretaker-compliance MUST satisfy every amendment.

There are no informative sections. Everything in this pack is binding.

Effect Summary

- Kernel becomes implementable — §A-1 fixes the evaluation pipeline.
- MutationGovernanceBinding becomes verifiable — §A-2 defines schema + verify procedure.
- outputHash becomes deterministic — §A-3 removes ambiguity over what is hashed.
- traceId becomes reproducible — §A-4 fixes ULID format + generation point.
- Revocation becomes real-time enforceable — §A-5 mandates linearizable reads.
- Compensation becomes bounded — §A-6 caps cascade depth at 3.
- Bootstrap becomes secure and reproducible — §A-7 specifies the ceremony.
- Spec becomes internally consistent — §A-8 collapses normative/informative split.
- Caretaker becomes defined — §A-9 names the role formally.
- Quorum windows become deterministic — §A-10 picks logical time + grace OR leader arbiter.
- Replay drift becomes resolvable — §A-11 mandates classification + handler.
- Deny-by-default becomes provable — §A-12 scopes the genesis exception.

§A-1

Enforcement Kernel — Formal Interface + Algorithm

What was broken in v1: the kernel was named but not specified at the interface or algorithm level. Implementers had to invent the evaluation order, which destroys determinism guarantees across deployments.

Lock: the kernel exposes exactly three methods and evaluates requests through a six-step deterministic pipeline. No step may be reordered, skipped, or added. The pipeline is the canonical reference for any /governance/evaluate implementation.

1.1 Internal Interface

```
EnforcementKernel:
  process(request: PolicyEvaluationRequest) -> PolicyEvaluationResponse
  synthesizeDecision(evaluation: PolicyEvaluationResponse) -> DecisionOutcome
  enforce(binding: MutationGovernanceBinding) -> EnforcementResult
```

1.2 Evaluation Pipeline (Deterministic)

```
def evaluate(request):
    assert request.traceId
    assert request.capability.tokenId

    # Step 1: Capability validation
    token = validate_token(request.capability.tokenId)
    if not token.valid:
        return deny("INVALID_TOKEN")

    # Step 2: Policy resolution
    policies = resolve_policies(request.constraints.policyRefs,
                                request.context.jurisdiction)

    # Step 3: Constraint evaluation
    results = []
    for p in policies:
        results.append(evaluate_policy(p, request))

    # Step 4: Hard constraint check
    if any(r.fail and r.type == "hard" for r in results):
        return deny("HARD_CONSTRAINT_FAIL")

    # Step 5: Quorum requirement derivation
    quorum = derive_quorum(request.action.type)

    # Step 6: Decision synthesis input
    return PolicyEvaluationResponse(
        decision = "allow" if all_pass(results) else "escalate",
        constraintResults = results,
        quorum = quorum,
```

```
        evaluationId = hash(request)
    )
```

1.3 Decision Synthesis

```
def synthesize(evaluation):
    if evaluation.decision == "deny":
        return DENY

    if not evaluation.quorum.satisfied:
        return ESCALATE

    return ALLOW
```

EXPANSION · Why this order is mandatory

- Capability validation precedes policy resolution so revoked tokens never trigger policy I/O.
- Hard-constraint check runs before quorum derivation so denied requests never consume approver attention.
- Synthesis is pure: it is a function of the evaluation result, not of external state — this is what makes replay deterministic.

§A-2

MutationGovernanceBinding — Canonical Schema

What was broken in v1: the binding was referenced as a precondition for execution but never schematized. Two implementations could legitimately produce non-interoperable binding shapes, breaking cross-node verification.

Lock: the binding has the canonical schema below. The verification procedure (Step 8 fix) is normative — a binding that fails any assertion is treated as a hard deny, not a soft warning.

2.1 Schema

```
MutationGovernanceBinding:
  bindingId: string
  traceId: string

  request:
    evaluationId: string

  capabilityToken:
    tokenId: string
    signature: string

  evaluation:
    decision: allow|deny|escalate
    constraintHash: string

  quorum:
    required: integer
    obtained: integer
    approvers: [string]

  decisionRecordRef:
    decisionId: string

  integrity:
    bindingHash: string
    signature: string
```

2.2 Verification Procedure (Step 8 Fix)

```
def verify_binding(binding):
    assert binding.traceId
    assert verify_signature(binding.integrity.signature)

    assert binding.quorum.obtained >= binding.quorum.required
    assert binding.evaluation.decision == "allow"

    assert hash(binding.request) == binding.evaluation.constraintHash
```

```
    return True
Failure -> hard deny
```

FAILURE · Any verification assertion failure = hard deny

Binding verification is fail-closed. There is no retry, no soft path, no escalation — the mutation is rejected and a DecisionRecord(type="binding_verification_failed") is emitted.

§A-3

outputHash — Canonical Definition

What was broken in v1: outputHash was referenced without a definition. Implementers variously hashed (a) response text, (b) request payloads, or (c) post-state. Replay never converged across deployments.

Lock: outputHash is SHA-256 of canonical-JSON over a fixed three-field object: post-mutation state, decision outcome, and resource id. Volatile fields are excluded.

3.1 Definition (Non-ambiguous)

```
outputHash = SHA256(
  canonical_json({
    "postState": state_after_mutation,
    "decision": decision_outcome,
    "resource": resource_id
  })
)
```

Rules

- MUST hash post-mutation state, not response text
- MUST use canonical serialization (RFC 8785 compliant)
- MUST exclude volatile fields (timestamps, signatures)

EXPANSION · Why these three fields and no others

postState captures what changed; decision captures why it was allowed; resource binds the hash to the addressable target. Adding actor or timestamp would break replay equivalence across nodes whose clocks drift; omitting any of the three would let two distinct mutations collide on hash.

§A-4

traceId — Required Format

What was broken in v1: traceId was mandatory but its format was unspecified. UUIDv4, hex strings, and even integer counters appeared in early implementations — none lexicographically sortable, breaking observability tooling.

Lock: traceId is a 26-character ULID in Crockford Base32. It is generated at request intake and propagates unchanged across the entire lifecycle.

4.1 Specification

```
traceId:
  type: ULID
  length: 26 chars
  encoding: Crockford Base32
  properties:
    - time-ordered
    - globally unique
    - lexicographically sortable
```

4.2 Generation Point

- MUST be generated at request intake
- MUST propagate unchanged across entire lifecycle

EXPANSION · Implementation note

Generate via the canonical ULID algorithm — 48-bit timestamp (ms since epoch) + 80 bits of cryptographic randomness, encoded in Crockford Base32. Reject any inbound request whose traceId fails format validation; do not attempt to repair or substitute.

§A-5

Revocation Propagation Architecture

What was broken in v1: token revocation was specified as a state change but its propagation model was undefined. A revoked token could remain usable for an unbounded window across the cluster.

Lock: revocation lives in a strongly-consistent distributed KV. Every evaluation queries it. Caches are bounded to 50ms. Tier 3 mutations require linearizable reads.

5.1 Model

```
RevocationSystem:
  store: distributed KV (strongly consistent)
  structure:
    tokenId -> status (active|revoked)
```

5.2 Propagation Mechanism

- Write path: quorum-signed revocation → commit to KV
- Read path: every /governance/evaluate MUST query KV
- Cache: max TTL = 50ms

5.3 Mid-flight Enforcement

```
if token.status == "revoked":
    abort_execution()
    emit DecisionRecord(type="revocation_enforced")
```

5.4 Consistency Requirement

- MUST use linearizable reads for Tier 3
- Tier 1–2 may use bounded-staleness reads (<50ms)

> INVARIANT · Revocation is real-time, not eventual
>
> A token revoked at T must be unusable for any Tier 3 mutation evaluated after T. The 50ms staleness budget applies only to Tier 1–2 reads.

§A-6

Compensation Cascade — Hard Limit

What was broken in v1: compensation depth was advisory. Pathological policy interactions could produce unbounded compensation chains, exhausting the ledger and masking the original fault.

Lock: the maximum compensation depth is 3. Exceeding it raises a constitutional fault, which is itself a non-recoverable event — it requires human caretaker intervention.

6.1 Rule (No longer optional)

```
MAX_COMPENSATION_DEPTH: 3
```

6.2 Enforcement

```
if compensation_depth(originalDecisionId) >= 3:
    raise ConstitutionalFault("COMPENSATION_LIMIT_EXCEEDED")
```

EXPANSION · Why 3 and not N

Three is empirically the deepest legitimate chain observed in correct policy sets: original mutation → compensating reversal → corrective re-application. Any chain longer than that is, by inspection of every audited deployment, either a policy bug or an adversarial pump. Capping the depth turns the bug into a loud, ledgered fault instead of silent ledger growth.

§A-7

Bootstrap Ceremony — Explicit Protocol

What was broken in v1: the genesis DecisionRecord was required but the ceremony that produces it was undefined. A solo operator on a network-connected machine could legitimately claim genesis, defeating the trust root.

Lock: bootstrap requires ≥2 authorities, HSM-backed key generation, an air-gapped environment, multi-party signatures, and external anchoring of the genesis hash before deny-by-default is enabled.

7.1 Requirements

```
BootstrapCeremony:
  participants: >= 2 authorities
  hardware:
    - HSM-backed key generation REQUIRED
  environment:
    - air-gapped REQUIRED
```

7.2 Steps

- Generate root GovernanceAuthority keypair (HSM)
- Create genesis DecisionRecord:
- type: GENESIS
- authority: root
- constraints: initial substrate
- Multi-party sign (≥2 signatures)
- Anchor hash externally (transparency log or equivalent)
- Distribute public key to all nodes
- Enable deny-by-default AFTER genesis commit

INVARIANT · No genesis without ceremony, no operation without genesis

The order is fixed: ceremony → anchored genesis → deny-by-default → first operational evaluation. Skipping any step invalidates the deployment.

§A-8

Structural Fix — Locked Spec Consistency

What was broken in v1: the manual referenced “supplement” and “extended” sections, which implied an informative tier. In a locked spec, informative content is a loophole — implementers can claim compliance while ignoring half the document.

Lock: there are no informative sections. Every numbered section in this manual is normative.

Rule

```
Sections:
  Normative: §§1–50 (ALL required)
  Informative: NONE
```

Remove “supplement/extended” classification. Everything is binding.

EXPANSION · How earlier wording is reinterpreted

The headers “Extended Supplement” (§§12–32) and “Final Supplement” (§§33–50) appear in v1 for navigational reasons only. Under v1.1 they carry no weight — every section under those banners is normative and required for caretaker compliance, identical in force to §§1–11.

§A-9

Caretaker — Formal Definition

What was broken in v1: the term “caretaker-compliant” appeared on the cover and in invariants, but “caretaker” was never defined. Without a definition there is no accountable role.

Lock: a caretaker is the authorized operator/maintainer/amender of a governed system, bound by the constitution. The role carries three explicit responsibilities.

```
Caretaker:
  definition: entity (human or system) authorized to operate,
              maintain, or amend a governed system within
              constitutional constraints

  responsibilities:
    - enforce invariants
    - manage authority lifecycle
    - execute governed amendments
```

EXPANSION · Caretaker vs. authority vs. operator

- Authority — holds a signing key recognized by the kernel. Cryptographic role.
- Operator — runs the infrastructure. Operational role, no constitutional power on its own.
- Caretaker — the union: an authorized entity whose actions are bounded by, and accountable to, the constitution. Every authority is a caretaker; not every operator is.

§A-10

Quorum Window Determinism

What was broken in v1: approval timing windows used wall-clock comparisons. Boundary approvals — those arriving exactly at window close — were resolved differently across nodes with skewed clocks, producing split-brain quorum decisions.

Lock: implementations MUST choose ONE of the two resolutions below and document the choice in their deployment manifest. The boundary case cannot remain undefined.

Fix

```
QuorumResolution:
  method: logical timestamp ordering
  rule:
    - approvals valid if timestamp <= window_end + 500ms grace
OR
leaderNode:
  authority: final arbitration on boundary conflicts
```

(Must choose one—cannot remain undefined)

EXPANSION · Choosing between the two

- Logical timestamps + 500ms grace — preferred for symmetric clusters where no node has special standing. Requires a logical clock (Lamport or hybrid).
- Leader arbiter — preferred when a leader already exists for other reasons (e.g. Raft). Single point of resolution; simpler reasoning at the cost of leader-availability dependence.
- Whichever you pick MUST be recorded in the deployment manifest and is itself subject to governed amendment.

§A-11

Replay Drift Resolution

What was broken in v1: the replay subsystem could detect drift between recorded and recomputed outputs but had no mandated handler. Drift events accumulated as warnings, masking both legitimate policy evolution and active tampering.

Lock: every drift event MUST be classified into one of three categories and routed to its handler. Drift cannot remain unhandled.

Required Procedure

```
ReplayDriftResolution:
  step1: flag drift event
  step2: classify (policy_change | tamper | ambiguity)
  step3:
    if policy_change:
        mark "expected drift"
    if tamper:
        trigger incident response
    if ambiguity:
        escalate to Arbiter
```

Drift cannot remain unhandled.

EXPANSION · Classifier inputs

- policy_change — the policy version active at original evaluation differs from the version active at replay; recorded constraintHash matches old version.
- tamper — inputs match, policies match, but recorded outputHash diverges from recomputed; the ledger has been mutated outside the kernel.
- ambiguity — inputs and policies match but evaluation produced non-deterministic intermediate state (e.g. external clock-dependent rule); requires arbiter ruling on which side is canonical.

§A-12

Deny-by-Default — Bootstrap Exception

What was broken in v1: deny-by-default was stated as universal, but the genesis DecisionRecord cannot itself be approved by an existing constitution — there is none yet. This created a circular dependency that some implementations resolved by quietly disabling deny-by-default during bootstrap.

Lock: the only exception to deny-by-default is the GENESIS record. The exception is scoped to that single record; once committed, deny-by-default applies universally and with no further carve-outs.

Explicit Rule

```
GenesisException:
  scope: single GENESIS DecisionRecord
  effect: bypass deny-by-default ONLY for root authority creation
After genesis:
  default: DENY
```

FAILURE · Any other carve-out is a constitutional fault

If a deployment disables deny-by-default for any reason other than the single GENESIS commit, it is non-compliant. Detection: audit for DecisionRecords with type != GENESIS that carry the bootstrap-bypass flag.

FINAL STATE · V1.1

With these corrections — the locked spec stands

With these corrections:

- Kernel becomes implementable
- Binding becomes verifiable
- Hashing becomes deterministic
- Revocation becomes real-time enforceable
- Bootstrap becomes secure and reproducible
- Spec becomes internally consistent

INVARIANT · v1.1 = v1 core + amendments §§A-1 through A-12

A deployment is caretaker-compliant under v1.1 if and only if it satisfies the v1 core spec AND every amendment in this pack. There are no partial conformance levels.

APPENDIX A

Glossary

Term	Definition
PER	Proposed Execution Request — actor + action + resource + mutation + traceId; the input to /governance/evaluate.
PEResult	PolicyEvaluationResult — outcome (allow/deny/escalate), policyRefs, basis.
DecisionRecord	Append-only, hash-linked, signed record of an evaluated mutation.
MGB	MutationGovernanceBinding — token + PEResult + DecisionRecord + quorum, required pre-execution.
CapabilityToken	Signed, scoped, attenuable authorization. No token = no mutation.
QuorumApproval	Individually signed, governance-evaluated approval bound to a token.
traceId	End-to-end correlation id; mandatory across all primitives.
Compensation	Forward-only corrective mutation referencing originalDecisionRecordId.
payloadHash / payloadRef	Content hash + immutable storage pointer for observability events.
recordHash	sha256 of canonical DecisionRecord including authoritySig.
prevRecordHash	Link to the prior DecisionRecord; chain break = halt.
Genesis record	First DecisionRecord; prevRecordHash = 0; root of the ledger.
Caretaker	Authorized entity (human or system) that operates, maintains, or amends a governed system within constitutional constraints (§A-9).
ConstitutionalFault	Non-recoverable runtime fault raised when an invariant is breached (e.g. compensation depth > 3, §A-6); requires human caretaker intervention.
RevocationSystem	Strongly-consistent distributed KV mapping tokenId → status; queried by every evaluation (§A-5).
GenesisException	The single, scoped carve-out that allows the GENESIS DecisionRecord to commit before deny-by-default takes effect (§A-12).
BootstrapCeremony	≥2-authority, HSM-backed, air-gapped procedure that produces the genesis DecisionRecord and the trust root (§A-7).

APPENDIX B

Primary References

- FIPS PUB 180-4, Secure Hash Standard (SHS), NIST. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf
- RFC 8032, Edwards-Curve Digital Signature Algorithm (EdDSA). https://www.rfc-editor.org/rfc/rfc8032
- RFC 8785, JSON Canonicalization Scheme (JCS). https://www.rfc-editor.org/rfc/rfc8785
- RFC 7519, JSON Web Token (JWT). https://www.rfc-editor.org/rfc/rfc7519
- NIST SP 800-207, Zero Trust Architecture. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-207.pdf
- NIST SP 800-53 Rev. 5, Security and Privacy Controls. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf
- RFC 5905, Network Time Protocol Version 4. https://www.rfc-editor.org/rfc/rfc5905
- RFC 6749, The OAuth 2.0 Authorization Framework. https://www.rfc-editor.org/rfc/rfc6749
- AICPA, SOC 2 Trust Services Criteria (TSC). aicpa-cima.com
- EU GDPR, Regulation (EU) 2016/679. eur-lex.europa.eu/eli/reg/2016/679/oj

FIELD MANUAL · UNIVERSAL GOVERNANCE SPEC v1.1 · CARETAKER-COMPLIANT · END